Distance Vector Routing (DVR) algorithm

```
#include <stdio.h>
#define INF 999
int main() {
   int cost[10][10],dist[10][10];
   printf("enter no of nodes");
   int n=0;
   scanf("%d",&n);
   printf("enter matrix:\n");
   for(int i=0;i<n;i++){
      for(int j=0;j<n;j++){
         scanf("%d",&cost[i][j]);
         if(cost[i][j]==0&& i!=j){
            cost[i][j]=INF;
         }
         dist[i][j]=cost[i][j];
      }
   }
   for(int k=0;k<n;k++){
      for(int i=0;i<n;i++){
         for(int j=0;j<n;j++){
            if(dist[i][j]>dist[i][k]+dist[k][j]){
               dist[i][j]=dist[i][k]+dist[k][j];
            }
         }
      }
   }
```

```
    for(int i=0;i<n;i++){
        printf("node:%d\n",i);
        for(int j=0;j<n;j++){
            printf("%d %d\n",j,dist[i][j]);
        }}
    return 0;
}
```

Output:

```
enter no of nodes4
enter matrix:
0 2 5 0
2 0 1 4
5 1 0 2
0 4 2 0
node:0
0 0
1 2
2 3
3 5
node:1
0 2
1 0
2 1
3 3
node:2
0 3
1 1
```

2 0

3 2

node:3

0 5

1 3

2 2

3 0

Dijkstra's Algorithm

```
#include<stdio.h>
#define INF 9999
#define N 10
int G[N][N], dist[N], pred[N], vis[N];

void dijkstra(int n, int s) {
   int i, j, cnt = 1, mn, next;
   for (i = 0; i < n; i++) {
      dist[i] = (G[s][i] ? G[s][i] : INF);
      pred[i] = s;
      vis[i] = 0;
   }
   dist[s] = 0; vis[s] = 1;
   while (cnt < n) {
      mn = INF;
      for (i = 0; i < n; i++)
         if (!vis[i] && dist[i] < mn) { mn = dist[i]; next = i; }
      vis[next] = 1; cnt++;
```

```
        for (i = 0; i < n; i++)
            if (!vis[i] && mn + (G[next][i] ? G[next][i] : INF) < dist[i]) {
                dist[i] = mn + (G[next][i] ? G[next][i] : INF);
                pred[i] = next;
            }
    }
    for (i = 0; i < n; i++) if (i != s) {
        printf("\n%d -> %d : cost=%d, path: ", s+1, i+1, dist[i]);
        j = i;
        while (j != s) { printf("%d ", j+1); j = pred[j]; }
        printf("%d", s+1);
    }
}

int main() {
    int n, s, i, j;
    printf("Vertices: "); scanf("%d", &n);
    printf("Adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &G[i][j]);
    printf("Start node: "); scanf("%d", &s);
    dijkstra(n, s - 1);
}
```

Output:

Vertices: 6

Adjacency matrix:

0 7 9 0 0 14

7 0 10 15 0 0

9 10 0 11 0 2

0 15 11 0 6 0

0 0 0 6 0 9

4 0 2 0 9 0

Start node: 1

1 -> 2 : cost=7, path: 2 1

1 -> 3 : cost=9, path: 3 1

1 -> 4 : cost=20, path: 4 3 1

1 -> 5 : cost=20, path: 5 6 3 1

1 -> 6 : cost=11, path: 6 3 1

Leaky Bucket

```
#include <stdio.h>

int main()
{
    int bucket_size, output_rate;
    int n, packet_size;
    int storage = 0;

    printf("Enter bucket capacity: ");
```

```
scanf("%d", &bucket_size);

printf("Enter output rate: ");
scanf("%d", &output_rate);

printf("Enter number of incoming packets: ");
scanf("%d", &n);

for (int i = 1; i <= n; i++)
{
   printf("\nEnter size of packet %d: ", i);
   scanf("%d", &packet_size);

   printf("Packet size: %d\n", packet_size);

   if (packet_size > bucket_size - storage)
   {
      printf("Bucket overflow! Packet rejected.\n");
   }
   else
   {
      storage += packet_size;
      printf("Packet accepted.\n");
      printf("Bytes in bucket: %d\n", storage);
   }

   if (storage > 0)
   {
```

```
            int sent = (storage < output_rate)? storage : output_rate;
            storage -= sent;
            printf("Bytes sent: %d\n", sent);
            printf("Bytes remaining: %d\n", storage);
        }
    }

    while (storage > 0)
    {
        int sent = (storage < output_rate)
                ? storage : output_rate;

        storage -= sent;

        printf("\nBytes sent: %d\n", sent);
        printf("Bytes remaining: %d\n", storage);
    }

    printf("\nAll packets processed.\n");

    return 0;
}
```

Output:

```
Enter bucket capacity: 10
Enter output rate: 4
Enter number of incoming packets: 3
```

Enter size of packet 1: 20

Packet size: 20

Bucket overflow! Packet rejected.

Enter size of packet 2: 5

Packet size: 5

Packet accepted.

Bytes in bucket: 5

Bytes sent: 4

Bytes remaining: 1

Enter size of packet 3: 5

Packet size: 5

Packet accepted.

Bytes in bucket: 6

Bytes sent: 4

Bytes remaining: 2

Bytes sent: 2

Bytes remaining: 0

All packets processed.

Link State Routing (LSR) algorithm

```
#include <stdio.h>
#define INF 999

int main()
{
    int n, src, graph[10][10];
    int dist[10], visited[10] = {0};

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if(i != j && graph[i][j] == 0)
                graph[i][j] = INF;
        }

    printf("Enter source node: ");
    scanf("%d", &src);

    for(int i = 0; i < n; i++)
        dist[i] = INF;

    dist[src] = 0;
```

```
    for(int k = 0; k < n; k++) {
        int u = -1, min = INF;

        for(int i = 0; i < n; i++)
            if(!visited[i] && dist[i] < min)
                min = dist[i], u = i;

        if(u == -1) break;

        visited[u] = 1;

        for(int v = 0; v < n; v++)
            if(!visited[v] && graph[u][v] != INF && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }

    printf("\nNode\tDistance\n");
    for(int i = 0; i < n; i++)
        printf("%d\t%d\n", i, dist[i]);

    return 0;
}
```

Output:

```
Enter number of nodes: 6

Enter adjacency matrix:

0 7 9 0 0 14
```

7 0 10 15 0 0

9 10 0 11 0 2

0 15 11 0 6 0

0 0 0 6 0 9

4 0 2 0 9 0

Enter source node: 1

Node	Distance
0	7
1	0
2	10
3	15
4	21
5	12

encrypt and decrypt

```
#include <stdio.h>
#include <ctype.h>

int main()
{
   char text[100], ch;
   int key, i;

   printf("Enter a string: ");
```

```
fgets(text, 100, stdin);

printf("Enter key: ");
scanf("%d", &key);

key = key % 26;

// Encryption
for(i = 0; text[i] != '\0'; i++) {
    ch = text[i];

    if(isupper(ch))
        text[i] = (ch - 'A' + key) % 26 + 'A';
    else if(islower(ch))
        text[i] = (ch - 'a' + key) % 26 + 'a';
}

printf("Encrypted: %s", text);

// Decryption
for(i = 0; text[i] != '\0'; i++) {
    ch = text[i];

    if(isupper(ch))
        text[i] = (ch - 'A' - key + 26) % 26 + 'A';
    else if(islower(ch))
        text[i] = (ch - 'a' - key + 26) % 26 + 'a';
}
```

```
    printf("Decrypted: %s", text);


    return 0;
}
```

Output:

Enter a string: yashwantt

Enter the key: 7

Encrypted string: fhzodhuaa

Decrypted string: yashwantt